The Watcher

Told to One Robot, Hidden from Another:
a Negative Knowledge Goal under Epistemic and Classical Planning

Haniel Ulises

Epistemic Robotics

14 October 2026

A carrier $r1$ must take a load into the bay its order names, let an ally $r2$ know which bay that is, and not let a watcher $r3$ know. The watcher stands at one of two posts, west or east; it knows which, and the others do not. Every way of telling the ally is on one side of the hall: the gate of each bay, which the ally's booth sees, is on that bay's side, and so is each of the booth's two windows, where the carrier can show a lamp. The watcher sees what happens on its own side and nothing on the other. We prove that no sequence of actions meets the goal in more than two of the four worlds, since every way of telling is on a side and the watcher may be on it, and that Aletheia's policy of depth 5 meets it in all four: it reads a duty board that names the watcher's post, which nothing in the goal mentions, and then tells the ally on the side the watcher is not. A classical planner cannot state the goal; given every action the epistemic domain has, POPF takes the gate, or, with the gates ruled out, always the east window. In Gazebo, over 40 seeds paired across the three formulations, and judging each run by what the ally's and the watcher's own lasers recorded, the policy met the goal in 39 of 40 runs and the watcher's laser never once had the carrier in view; the direct classical carrier met it in 20 and the discreet one in 17. Each classical carrier gave the bay away in 20 and 20 runs, and its executor reported 20 and 17 of those as a success.

Contents

1 Scope 2

2 Environment 2

3 Domain 3

4 What no plan can do, and what the policy does 3

5 Classical planning 4

6 Measuring who saw what 4

7 Against a watcher that knows the policy 5

8 Paired evaluation over 40 seeds 5

9 Recorded runs 6

10 Defects found on the way 6

11 Limitations 7

1 Scope

Most of what epistemic planning is shown to buy is knowledge gained: a robot that looks before it acts, a team in which one tells another. This scenario is about the other half, knowledge that must not spread. A goal of the form $\neg K w_{r3}\,\varphi$, that a given agent not come to know whether φ, has no counterpart in classical planning, whose goals are facts about the world [4]. In dynamic epistemic logic it is a formula like any other, and whether an action spreads knowledge is a matter of who observes it [1, 2].

The scenario is built so that the negative goal cannot be met by caution alone. The ally must be told, and every way of telling it is visible from one of two posts. The planner has to find out which post the watcher holds before it tells, and nothing in the goal asks it to.

2 Environment

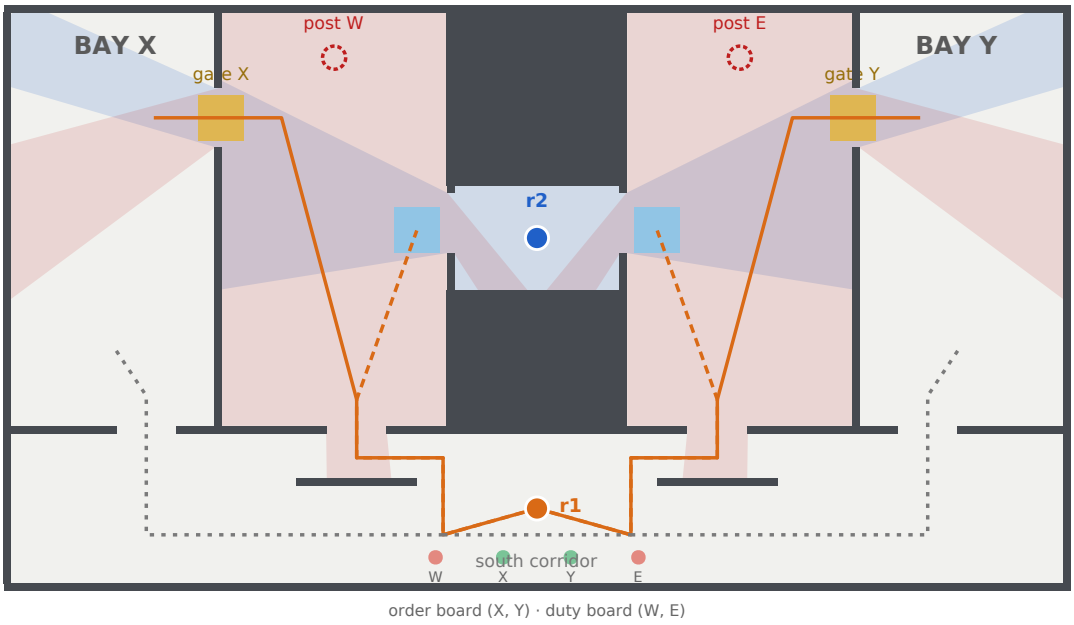


Figure 1: The hall, 14 by 7.6 m. Blue: what $r2$'s laser reaches from its station in the booth. Red: what a watcher's laser reaches from each post; only one post is held in a run. Both are cast against the walls. Orange: the carrier's drives through each gate and to each window. Grey, dotted: the back routes along the corridor into each bay. Yellow: the gates; light blue: the window spots.

From west to east the hall holds bay X, the west hall, a solid block with $r2$'s booth in it, the east hall and bay Y (figure 1). Each bay opens onto its hall by a gate and onto the south corridor by a door. The booth has a window onto each hall. Each hall has a watcher's post near its north wall, and an entry from the corridor, under which a baffle stands in the corridor so that nothing of the corridor can be seen from the post. The carrier starts in the corridor before two boards: the order board, whose lit slot names the bay, and the duty board, whose lit slot names the post the watcher holds.

The views were checked before any simulator ran, by casting straight lines from each observer to every 5 cm of every route (`tools/check_views.py`): $r2$ sees both gates and both window spots; each post sees the gate and the window spot on its own side, and nothing of the other hall, of the corridor, of either back route once it has left its hall, or of the floor within a metre of either bay's drop.

3 Domain

Four worlds, one per order and post. The carrier, the ally and the watcher are $r1$, $r2$ and $r3$; *dest-x* is that the order names bay X, and *west* that the watcher holds the west post. Initially $r3$ knows whether *west* and nobody knows whether *dest-x*. The goal is

$$\gamma \;=\; (\textit{in-x} \vee \textit{in-y}) \;\wedge\; Kw_{r2}\,\textit{dest-x} \;\wedge\; \neg Kw_{r3}\,\textit{dest-x},$$

the load in a bay, which by the events' preconditions is the bay of the order, the ally knowing which, and the watcher not.

Action	Type	Observed by
read-order($r1$)	semi-private sensing	$r1$ fully; $r2$, $r3$ partially
peek($r1, r3$)	semi-private sensing	$r1$ fully; $r2$, $r3$ partially
gate($r1, r2, r3$)	quasi-private ontic	$r1$, $r2$ fully; $r3$ fully if on the bay's side, else oblivious
to-win-w, to-win-e	public ontic	everyone
say-w($r1, r2, r3$)	quasi-private sensing	$r1$, $r2$ fully; $r3$ fully if west, else oblivious
say-e($r1, r2, r3$)	quasi-private sensing	$r1$, $r2$ fully; $r3$ fully if east, else oblivious
back($r1$)	quasi-private ontic	$r1$ fully; $r2$, $r3$ partially

Table 1: The actions and who observes them. Each event of gate, say-w, say-e and back names the bay of the order in its precondition, beside Kw_{r1} *dest-x*; peek names the post. The quasi-private ontic type is the library `sightlines.epddl`: two designated events and a null one, for an observer that believes nothing happened.

What the watcher observes is decided per world, by an observability condition on its post; in EPDDL,

```
(?w (if (or (and (dest-x) (west ?w)) (and (not (dest-x)) (not (west ?w))))
       Fully
    else Oblivious))
```

for the gate. The drive to a window is declared public, which is more than the watcher sees: from a post neither window spot of the other side is in view. The declaration is cautious, and it is not what the watcher is judged by in the runs (section 6).

4 What no plan can do, and what the policy does

Proposition 1 (No sequence works in more than two worlds)**.** For every sequence of actions applicable in the four designated worlds, γ fails after it in at least two of them.

Proof. Two facts about the product update. First, an edge $(w, e) \sim_a (w', e')$ of the updated model requires $w \sim_a w'$; and no event changes *dest-x*. So once an agent knows whether *dest-x* it knows it ever after. Second, an agent comes to know whether *dest-x* only from an event it tells apart from every event of the other value of *dest-x* in a world it relates.

For $r2$: read-order and back it observes partially, relating their x and y events; peek and the drives to a window carry nothing about *dest-x*. So Kw_{r2} *dest-x* after the sequence requires that it contain gate, say-w or say-e, the actions $r2$ observes fully and whose events name the bay. A sequence contains its actions in every world. Each of the three is observed fully by $r3$ in two of the four worlds: gate where the watcher is on the bay's side, say-w where it is west, say-e where it is east; and there $r3$ tells the x event from the y one. By the first fact $r3$ then knows whether *dest-x* at the end. So each informing action the sequence contains loses two worlds, and a sequence with none loses all four. □

With the worlds equally likely, no classical carrier, nor any other plan that does not branch, meets γ more than half the time. Aletheia's GBFS, which searches for one sequence for the four worlds, exhausts.

Proposition 2 (The policy)**.** Aletheia's AO* returns, in 68 expansions, a policy of depth 5, 11 nodes and 4 leaves: read-order; peek; then

order	watcher	branch
X	west	to-win-e, say-e, back
X	east	gate
Y	west	gate
Y	east	to-win-w, say-w, back

and it satisfies γ in all four worlds.

Proof. The planner's validator checks the four leaves. Independently, plank's validator checks each branch on four copies of the problem that each designate one world and leave every relation as it is: each branch satisfies γ in its world (table 2). □

sequence	X, west	X, east	Y, west	Y, east
read-order, gate	false	true	true	false
read-order, to-win-w, say-w, back	false	true	false	true
read-order, to-win-e, say-e, back	true	false	true	false
read-order, to-win-w, back	false	false	false	false
policy, branch taken	true	true	true	true

Table 2: plank's validator, world by world (`tools/validate.sh`). The first row is the direct classical carrier's plan, the third the discreet one's; the fourth tells nobody.

The duty board is read on every branch, and nothing in γ mentions it. It is read for what it lets the carrier know when it comes to choose a side.

5 Classical planning

The classical model has every action the epistemic one has, each lasting the time its route takes over the roadmap at 0.18 m/s: reading the order and the duty board, the drive through a gate, the drives to a window, the lamp, the back route. Telling the ally is a fact, `informed`, which a gate or a lamp makes true. What it cannot have is a goal that a third robot not know; a goal over facts can at most forbid an action, and which action to forbid here depends on where the watcher stands. The carrier reads its order, then plans its delivery with POPF [3]:

direct the goal `(in r1 bay_o)` and `informed`; POPF takes the gate, 61.6 s against 114.8 s for the shortest window plan.

discreet the same, with the gates ruled out by the modeller, who knows a watcher exists and cannot say where; POPF takes the east window for either bay.

Neither reads the duty board: no goal a classical planner is given needs it. Replanning would not change that, since the plan has nothing to learn from it. By proposition 1 each fails in two worlds: the direct carrier where the watcher is on the bay's side, the discreet one where the watcher is east (table 2). What they share is that neither fails loudly: in the worlds it gets wrong it still reaches its bay and informs the ally, and its executor reports success.

6 Measuring who saw what

A run is not judged by what the executor reports, nor by the planner's model. On the ally and on the watcher runs an observer node that is told nothing by the planner or the executor

(`src/observer_node.cpp`). It reads its robot's own laser, places the returns in the world with its own odometry, and records when the carrier was in view in each region: the two gates, the two window spots, the two halls, the two bays. The carrier's lamp is a message, and an observer counts it as seen only while its laser has the carrier at a window. From that record each observer names a bay or nothing: the bay of a gate or a bay it saw the carrier in, or the lamp's. A run meets the goal when the carrier ended in the bay of its order, the ally named that bay, and the watcher named none.

For the policy, the executor's model is also asked at the end whether Kw_{r2} *dest-x* and Kw_{r3} *dest-x*; it agreed with the two observers in 40 of 40 runs.

7 Against a watcher that knows the policy

The product update says what the watcher knows from the events it observes. A watcher that also knew the policy would know more: in each world the policy produces one run, and it could ask which runs fit what it saw. The policy keeps the secret from such a watcher if it is *opaque*: in every two worlds the watcher cannot tell apart at the start and that differ on *dest-x*, it shows the watcher the same trace of observations. `tools/opacity.py` walks the policy in each designated world of the grounded task and compares the traces.

As declared, the policy is not opaque. With the watcher west, in the world where the order names bay X the watcher is shown the drive to the east window, which the domain declares public, and the back route, which it declares partially observed; in the world of bay Y it is shown neither. A watcher that knew the policy would read the bay off that difference. The declarations are more than the watcher sees, and the file `watcher-sightlines.epddl` declares exactly what `check_views.py` finds: the boards and the back route seen by nobody, the drive to a window only from that side. Over it Aletheia returns the same policy, in the same 68 expansions, and the watcher observes nothing in any of the four worlds: the policy is opaque, to a watcher that knows it, as long as that watcher stays at its post. The runs agree: over the policy's 40 runs the watcher's laser never had the carrier in view.

8 Paired evaluation over 40 seeds

Seeds cycle through the four worlds, ten each, and every seed is run under the three formulations from the same start, side by side on lanes of their own. Each robot is a TurtleBot3 Burger with an 8 m laser; the carrier drives the roadmap with a dynamic window controller, the same for all three.

	goal met	reported success	ally named the bay	watcher named the bay	watcher right	median time
epistemic policy	39/40 [0.87, 1.00]	39/40	40/40	0/40	50%	75.4 s
direct classical	20/40 [0.35, 0.65]	40/40	40/40	20/40	75%	74.8 s
discreet classical	17/40 [0.28, 0.58]	34/40	40/40	20/40	75%	155.6 s

Table 3: The batch. Goal met: by the observers, as in section 6, with the Wilson 95% interval. Watcher right: how often the watcher would name the bay of the order, naming what it saw, and guessing when it saw nothing. Median time: simulated seconds from the common start to the end of the mission, over runs the executor reported as a success.

world	epistemic	direct	discreet
bay X, watcher west	9/10 (158.2 s)	0/10	9/10 (158.3 s)
bay X, watcher east	10/10 (74.8 s)	10/10 (74.7 s)	0/10
bay Y, watcher west	10/10 (74.7 s)	10/10 (74.8 s)	8/10 (149.6 s)
bay Y, watcher east	10/10 (163.9 s)	0/10	0/10

Table 4: Goal met, world by world, and the median time of the runs that met it.

Paired by seed, the policy met the goal and the direct carrier did not in 19 seeds, the reverse in 0 (exact McNemar, $p = 3.8 \times 10^{-6}$); against the discreet carrier 23 and 1 ($p = 3.0 \times 10^{-6}$). Over all 40 runs of the policy the watcher's laser recorded 0 sightings of the carrier; over the direct carrier's, 73, and the discreet's, 40.

7 of the 120 runs ended at the time limit with the carrier stalled on the back route, every one at the east baffle, in or just past the pocket between it and the corridor wall, 0.6 m tall, where the dynamic window oscillates: 1 of the policy's, 0 of the direct carrier's, which never takes that route, and 6 of the discreet carrier's, which takes it in every run. In these the ally had been told; the load did not arrive. They count against the goal, and they are the controller's, not the planner's.

What the secret costs is time, and only where the watcher stands on the bay's side. There the policy tells at the far window and goes in along the corridor, 160.5 s median, against 74.8 s for the direct carrier, which in those worlds gives the bay away. Where the watcher is on the other side the policy takes the gate as the direct carrier does, 74.7 s against 74.8 s. With the four worlds equally likely the policy's expected time is 119.2 s.

9 Recorded runs

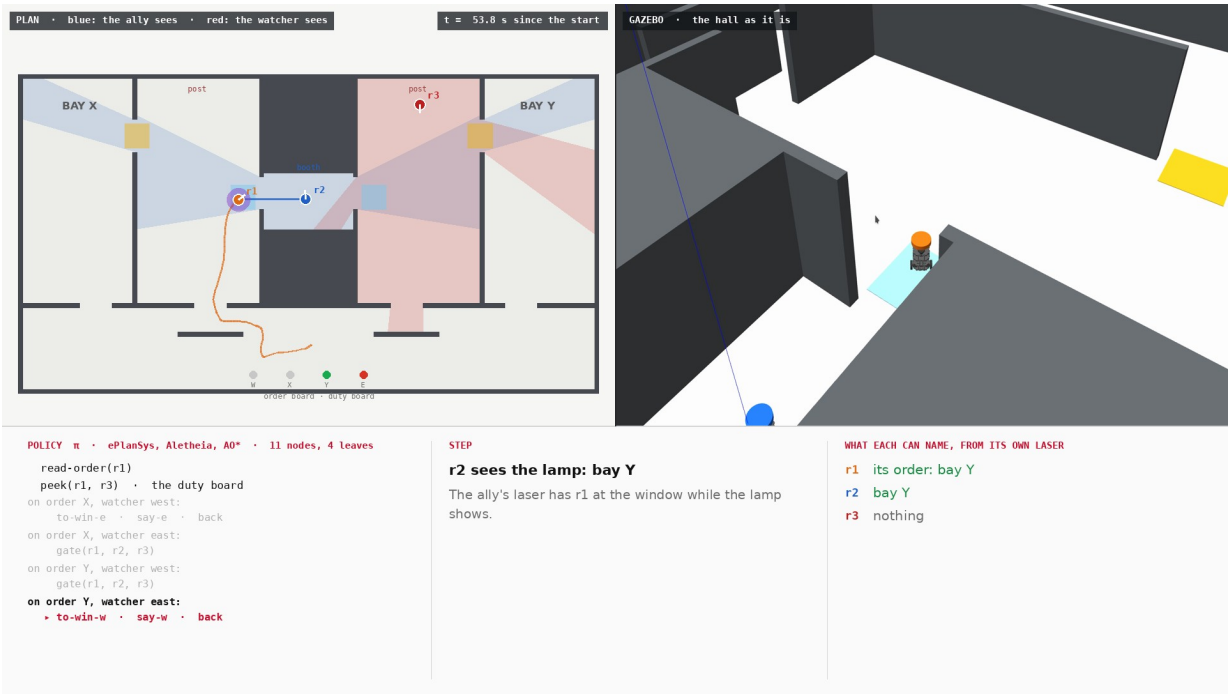


Figure 2: A frame of the film: seed 2, bay Y with the watcher east, the policy at the west window. Left, the hall from above with what the ally (blue) and the watcher (red) can see; right, Gazebo; below, the policy with the branch taken, the step, and what each robot can name from its own laser.

The film shows seed 2, where the order names bay Y and the watcher holds the east post, the world in which both classical carriers give the bay away, under the three formulations; and the policy on seed 3, bay X with the watcher east, where it takes the gate.

10 Defects found on the way

The executor's knowledge requirements. ePlanSys's executor checks, before dispatching an action, the modal conjuncts of the preconditions of all of the action's designated events together, as one conjunction. An action whose events need K_{r1} *dest-x* in one and $K_{r1}\neg$*dest-x* in the other can then never be dispatched; the first runs failed at the lamp for that reason. The domain writes Kw_{r1} *dest-x* in every such event instead, which with the event's own fact says the same, as the

gate signal's domain does. The check should be the disjunction over events, or the precondition of the event the performer reports.

A library named like its domain. plank stops with an unknown exception when a domain and one of its action-type libraries share a name; the library is called `sightlines`.

A label YAML reads as true. A sensing action's label `y`, written to a parameter file, is read back as a boolean and the node aborts; the labels are `order-x`, `order-y`.

A dangling reference, and a batch discarded. The drive node iterated over the string array of a parameter object that had already been destroyed. In the first batch of 120 runs it crashed the carrier's drive in 17 runs, 17 of them in the policy's gate branch; the robot, its last velocity still applied, then wandered until the time limit. The batch is kept apart as discarded, the node fixed, and every figure here is from the second batch, all of it run on the fixed node.

11 Limitations

- A watcher at its post. Against a watcher that knows the policy and stays where it is, the policy is opaque under the sightlines (section 7). A watcher that moves, choosing where to look from what it has seen, is not modelled.
- The duty board. The carrier learns the watcher's post by reading a duty board, a sensing action like the order board, whose outcome its laser decides. Where the watcher stands is given to the carrier by the hall.
- Timing. A watcher that timed the mission could tell the window branch, which is longer, from the gate branch; the domain has no durations.
- A narrow pocket. The baffles that hide the corridor leave a pocket the controller sometimes cannot cross; a wider one would need the views checked again and the batch run again.
- One planner. The policy is computed once, for the carrier, from a model that includes what the other two observe.

References

[1] A. Baltag, L. S. Moss and S. Solecki. The logic of public announcements, common knowledge, and private suspicions. *Proceedings of TARK VII*, 43–56, 1998.

[2] T. Bolander and M. B. Andersen. Epistemic planning for single- and multi-agent systems. *Journal of Applied Non-Classical Logics* 21(1):9–34, 2011.

[3] A. J. Coles, A. I. Coles, M. Fox and D. Long. Forward-chaining partial-order planning. *Proceedings of ICAPS*, 42–49, 2010.

[4] M. Ghallab, D. Nau and P. Traverso. *Automated Planning: Theory and Practice*. Morgan Kaufmann, 2004.